

Socioeconomic Factors in Voting Patterns

Monish Sinha [25283945]
James Kavanagh [16515816]

Cormac Kehoe [23411302]
Cole Ternullo [23202375]

Big Data, University College Dublin
Professor: Dr. Dylan-Ennis

Code Repository: <https://github.com/SinhaM101/mobility-credit>

Section 1. Introduction

This project draws on two primary data sources that together provide a rich longitudinal view of socioeconomic conditions and political outcomes across the United States. Our first source is the U.S. Census American Community Survey (ACS) 5-Year Data Profiles, which cover social, economic, housing, and demographic characteristics for more than 3,100 U.S. counties from 2009 to 2023. These data consist of over 3,060 CSV files totaling approximately 150 MB across four profile tables (DP02–DP05), with more than 500 variables per county capturing detailed measures of income, employment, education, housing, and population composition. Our second source is a county-level presidential election dataset spanning elections from 2000 to 2024, comprising 94,019 records across seven election cycles and reporting vote counts by candidate and party.

Volume

We process a substantial volume of structured data drawn primarily from the ACS and U.S. presidential election results. The ACS component comprises approximately 150 MB distributed across 3,060 CSV files (51 states × 15 years × 4 tables), including four Data Profile tables: Economic characteristics (DP03, ~40 MB with 141 variables per county), Social characteristics (DP02, ~35 MB with 158 variables), Demographic characteristics (DP05, ~20 MB with 94 variables), and Housing characteristics (DP04, ~25 MB with 145 variables). We combine these data with presidential voting records totaling 8.9 MB and 94,019 rows, resulting in approximately 160 MB of structured data overall. Our merged dataset spans more than 3,143 U.S. counties over 15 years and four ACS profiles, yielding 47,141 county-year observations with up to 400 columns each. All data processing is conducted on a macOS system (MacBook Air M4, 24 GB RAM) using Python 3.9.6 and Pandas 2.3.3.

Big Data Justification: While 160 MB may seem modest in absolute terms, the dataset qualifies as "big data" relative to our hardware constraints. With 47,141 rows × 400 columns, the in-memory DataFrame consumes ~67 MB, and join operations require temporary copies that push peak memory to 200+ MB. On a system with 24 GB RAM, this leaves limited headroom for parallel workers. More critically, the 3,060 individual CSV files create I/O bottlenecks that single-threaded Pandas cannot efficiently handle—a classic "variety" challenge that MapReduce is designed to address through distributed file reads.

Variety

We integrate two datasets with fundamentally different structures and analytical roles, demonstrating meaningful data variety. The ACS Data Profiles (DP02, DP03, DP04, and DP05) are stored in a wide format with hundreds of variables per county, allowing us to capture detailed socioeconomic indicators such as income, employment, education, housing conditions, and demographic composition. In contrast, the presidential voting dataset is organized in a long format with many rows, reporting election outcomes by county, candidate, party affiliation, and vote counts across multiple election cycles. These structural differences reflect distinct purposes: the ACS data provide rich contextual and explanatory variables, while the voting data represent political outcomes. Together, they enable us to conduct correlation and regression analyses that link socioeconomic conditions to electoral behavior.

Objective and Value

Our objective is to analyze how county-level economic conditions correlate with voting patterns in U.S. presidential elections from 2000 to 2024. By examining relationships between factors such as income inequality, employment rates, and electoral outcomes, we aim to better understand how socioeconomic conditions shape political behavior at a granular geographic level. This analysis adds value by helping us identify underserved communities and regional disparities, while also informing policy discussions by highlighting which economic indicators most strongly predict political preferences across diverse U.S. counties.

Section 2. Traditional Solution

Before developing the big data pipeline, we built a single-threaded Python prototype to validate the processing logic and establish performance baselines. This prototype uses no parallelism and processes data using the Pandas library.

Environment Setup

Development Environment:

- Hardware: MacBook Air M4 with Apple Silicon, 24 GB unified memory, 512 GB SSD
- Operating System: macOS Sequoia 15.3
- Python Version: 3.9.6 (system default)
- Key Libraries: Pandas 2.3.3, NumPy 1.24.0, PySpark 4.0.2

The prototype was developed in a Jupyter notebook environment for interactive exploration, then refactored into standalone Python scripts for reproducibility. All timing measurements use Python's `time.time()` for wall-clock timing, and memory profiling uses `tracemalloc` for peak memory tracking.

Data Sources and Structure

ACS Economic Data (DP03):

The American Community Survey Data Profile 03 contains economic characteristics including employment status, occupation, industry, income, and poverty statistics. Each row represents one county, with 144 columns capturing detailed economic indicators. The data is pre-merged from 51 state-level files into a single CSV for the year 2020. Key columns include:

- `state_fips`, `county_fips`: Geographic identifiers
- `employment_status_population_16_years_and_over`: Working-age population
- `income_and_benefits_median_household_income`: Median household income
- `percentage_below_poverty_level`: Poverty rate

Presidential Voting Data:

County-level election results from MIT Election Data + Science Lab, containing vote counts by candidate, party, and county for all presidential elections 2000-2024. The data is in long format with 94,019 rows (multiple candidates per county per election). Key columns include:

- `year`: Election year (2000, 2004, 2008, 2012, 2016, 2020, 2024)
- `county_fips`: 5-digit FIPS code
- `candidate`, `party`: Candidate name and party affiliation
- `candidatevotes`, `totalvotes`: Vote counts

Processing Pipeline Overview

The traditional solution follows a five-step ETL (Extract, Transform, Load) pipeline:

1. Extract: Load raw CSV files into Pandas DataFrames
2. Transform: Filter by year, standardize FIPS codes, identify election winners
3. Load: Merge datasets on common key (FIPS code)
4. Analyze: Aggregate statistics by winning party
5. Output: Generate summary tables and visualizations

This pipeline is entirely sequential—each step must complete before the next begins. The CPU utilization during execution shows a single core at 100% while other cores remain idle, demonstrating the limitation of single-threaded processing.

Step 1: Load ACS Economic Data

Load the consolidated ACS DP03 economic data for 2020 containing employment, income, and poverty statistics for all U.S. counties.

Code:

```
import pandas as pd
import time
import tracemalloc

tracemalloc.start()
start = time.time()
econ = pd.read_csv('data/acs_merged/economic/economic_2020.csv')
step1_time = time.time() - start
```

Execution Results: Rows: 3,143 | Columns: 144 | Time: 0.024s | Memory: 12.8 MB

Step 2: Load Presidential Voting Data

Load county-level presidential election results containing vote counts by candidate and party for all elections from 2000-2024.

```
start = time.time()
pres = pd.read_csv('data/countypres_2000-2024.csv')
step2_time = time.time() - start
```

Execution Results: Rows: 94,019 | Columns: 13 | Time: 0.045s | Memory: 39.1 MB

Step 3: Filter and Clean Data

Filter presidential data to match the economic data year (2020) and create standardized 5-digit FIPS codes for merging. FIPS codes are county identifiers (2-digit state + 3-digit county).

```
pres_2020 = pres[pres['year'] == 2020].copy()
econ['full_fips'] = econ['state_fips'].astype(str).str.zfill(2) + \
                    econ['county_fips'].astype(str).str.zfill(3)
pres_2020['full_fips'] = pres_2020['county_fips'].astype(str).str.zfill(5)
```

Execution Results: Filtered Rows: 22,093 | Time: 0.012s

Step 4: Merge Datasets

Join the economic data with presidential voting data on the county FIPS code, creating a unified dataset.

```
merged = pd.merge(econ, pres_2020[['full_fips', 'party', 'candidatevotes']],
                  on='full_fips', how='inner')
```

Execution Results: Merged Rows: 21,894 | Columns: 148 | Time: 0.008s | Memory: 67.0 MB

Step 5: Aggregate and Analyze

Determine the winning party per county (highest vote count) and calculate average economic indicators grouped by winning party.

```
winner_idx = pres_2020.groupby('full_fips')['candidatevotes'].idxmax()
winners = pres_2020.loc[winner_idx][['full_fips', 'party']]
analysis = pd.merge(econ, winners, on='full_fips')
result = analysis.groupby('winning_party').agg({'pop_col': ['mean', 'sum', 'count']})
```

Execution Results: Time: 0.008s

Analysis Results (2020 Election):

Party	County Count	Total Population	Avg Population
DEMOCRAT	546	157,753,458	288,926
REPUBLICAN	2,569	103,559,736	40,311

Execution Summary:

Step	Description	Time (s)
1	Load Economic Data	0.024
2	Load Presidential Data	0.045
3	Filter & Clean	0.012
4	Merge Datasets	0.008
Total		0.089

Peak Memory Usage: 67.0 MB. Step 2 (Load Presidential Data) consumes 51% of total execution time.

Limitations of Traditional Approach

While the prototype successfully produces correct results, it exhibits several limitations that become critical at scale:

- Single-Threaded Execution:
The entire pipeline runs on a single CPU core. On our 8-core M4 processor, this means 87.5% of available compute capacity sits idle. For I/O-bound operations like CSV parsing, this is particularly wasteful since the CPU spends most of its time waiting for disk reads.
- Memory Pressure:
Pandas loads entire datasets into memory before processing. With 67 MB peak usage for just 2020 data, scaling to all 15 years (2009-2023) would require ~1 GB of RAM. Join operations create temporary copies, potentially doubling memory requirements. On systems with limited RAM, this leads to swap thrashing.

No Fault Tolerance:

If the process crashes mid-execution (e.g., out of memory), all progress is lost. There is no checkpointing or recovery mechanism. For long-running jobs on large datasets, this is unacceptable.

Sequential I/O:

Reading 3,060 individual CSV files sequentially creates an I/O bottleneck. Modern SSDs can handle parallel reads efficiently, but single-threaded Pandas cannot exploit this capability.

Motivation for MapReduce

The limitations above motivate a transition to a distributed computing framework. MapReduce, implemented via Apache Spark, addresses each limitation:

- **Parallelism:** Spark distributes work across all available cores (and machines in a cluster)
- **Memory Management:** RDDs (Resilient Distributed Datasets) spill to disk when memory is exhausted
- **Fault Tolerance:** Spark tracks RDD lineage and can recompute lost partitions automatically
- **Parallel I/O:** Each Spark executor reads its assigned file partitions independently

The key insight is that our workload is "embarrassingly parallel" at the file level—each of the 3,060 ACS files can be processed independently before the final aggregation. This is the ideal use case for MapReduce: independent map operations followed by a shuffle and reduce.

Section 3. MapReduce Optimization

Identifying Bottlenecks

From the baseline analysis, we identified two time-consuming steps suitable for MapReduce optimization:

1. Step 2: Load Presidential Voting Data (0.045s, 51% of total time)
2. Step 4: Merge Datasets (0.008s, 9% of total time)

Why MapReduce is Suitable

These steps are suitable for parallel processing because:

1. **Data Loading (Map Phase):** Each CSV file is independent and can be read in parallel—an "embarrassingly parallel" pattern where each mapper processes a separate file.
2. **Merge/Join (Reduce Phase):** The merge operation groups records by county FIPS code. This is a classic reduce operation where all records with the same key are collected and combined.

Expected Improvement: 4–8× speedup on a multi-core system.

MapReduce Solution

We implemented the solution using Spark Core RDD API (NOT DataFrame/SQL) as required.

Map Function for Economic Data:

```
def parse_economic_line(line, header_indices):  
    """Map: Parse CSV line and emit (fips, record) pair"""  
    fields = line.split(',')  
    state_fips = fields[header_indices['state_fips']].zfill(2)  
    county_fips = fields[header_indices['county_fips']].zfill(3)  
    full_fips = state_fips + county_fips  
    return (full_fips, {'source': 'economic', 'population': fields[5]})
```

Map Function for Voting Data:

```
def parse_voting_line(line):  
    """Map: Parse CSV line and emit (fips, record) pair"""  
    fields = line.split(',')  
    county_fips = fields[4].replace('.0', '').zfill(5)
```

```

votes = int(float(fields[8])) if fields[8].strip() else 0
return (county_fips, {'source': 'voting', 'party': fields[7], 'votes': votes})

```

Reduce Function:

```

def merge_records(records):
    """Reduce: Merge economic and voting records for a county"""
    economic_data, voting_records = None, []
    for record in records:
        if record['source'] == 'economic': economic_data = record
        else: voting_records.append(record)
    winner = max(voting_records, key=lambda x: x['votes'])
    return {'population': economic_data['population'], 'winning_party': winner['party']}

```

Spark RDD Implementation:

```

from pyspark import SparkContext, SparkConf

conf = SparkConf().setAppName("ACS_Voting").setMaster("local[4]")
sc = SparkContext(conf=conf)

# MAP PHASE: Load and parse data
economic_rdd = sc.textFile("economic_2020.csv").map(parse_economic_line).filter(lambda x: x)
voting_rdd = sc.textFile("countypres.csv").map(parse_voting_line).filter(lambda x: x)

# SHUFFLE & REDUCE PHASE: Union and group by key
merged_rdd = economic_rdd.union(voting_rdd).groupByKey().mapValues(merge_records)

# AGGREGATE: Count by winning party using reduceByKey
party_counts = merged_rdd.map(lambda x: (x[1]['winning_party'], 1))\
    .reduceByKey(lambda a, b: a + b).collect()

```

Shuffle Cost Note: The groupByKey() operation triggers a full shuffle, moving all records with the same FIPS key to the same partition. This is the most expensive operation in the pipeline—network I/O and disk spill dominate execution time. For our 47K-row dataset, shuffle writes ~12 MB to disk. At scale (millions of rows), this shuffle cost would dwarf the map phase, making partitioning strategy critical. We use reduceByKey() for the final aggregation, which performs local combining before shuffle, reducing network traffic.

MapReduce Results

Small Dataset (2020 only, ~3MB):

Configuration	Time (s)	Speedup
Pandas CSV	0.094	1.00×
Spark RDD local[1]	1.730	0.05×
Spark RDD local[2]	0.935	0.10×
Spark RDD local[4]	0.653	0.14×

Large Dataset (All Years Merged, ~84MB, 47,141 rows):

Configuration	Time (s)	Speedup
Pandas All Years	2.768	1.00×
Spark RDD local[1]	1.427	1.94×
Spark RDD local[4]	0.776	3.57×

Analysis of Results

Small Dataset (2020 only):

- Spark RDD with 4 cores: 0.653s vs Pandas 0.094s (0.14× - slower)
- JVM startup overhead (~1-2s) dominates for small datasets

Large Dataset (All Years Merged, 84MB):

- Spark RDD with 4 cores: 0.776s vs Pandas 2.768s (3.57× speedup)
- Even with 1 core: 1.427s vs Pandas 2.768s (1.94× speedup)
- MapReduce benefits become clear with larger data volumes

Key Insight: MapReduce overhead is amortized over larger datasets. For datasets >50MB, Spark RDD provides significant speedup. The crossover point is approximately 30-50MB.

Projected Performance at Scale

Dataset Size	Pandas (est.)	Spark 4-core	Speedup
67 MB	0.089s	0.537s	0.17×
670 MB	0.89s	0.8s	1.11×
6.7 GB	8.9s	3.5s	2.5×
67 GB	89s	15s	5.9×

Conclusion

The MapReduce implementation is correct and functional, producing identical results to the baseline. However, the current dataset is too small to benefit from parallel processing—the overhead of Spark's distributed computing framework exceeds the computation time. For production use with larger datasets (>1 GB), the MapReduce approach would provide significant speedup.

Key Findings:

- Democratic-winning counties: 546 counties, avg population 288,926 (urban areas)
- Republican-winning counties: 2,569 counties, avg population 40,311 (rural areas)
- This quantifies the urban-rural political divide in the 2020 U.S. presidential election